PYTHON OBJECT-ORIENTED PROGRAMMING

Corrected Study Guide for RAG Retrieval

This version restores the missing Section 2, uses consistent section numbering, and includes clear definitions, runnable examples, and revision notes so the retrieval index can identify individual OOP concepts.

Contents

- 1. Introduction to OOP
- 2. Classes and Objects
- 3. Constructor and self
- 4. Encapsulation
- 5. Inheritance
- 6. Polymorphism
- 7. Abstraction
- 8. Class vs Instance Variables
- 9. Special Methods
- 10. Composition
- 11. Interview Revision

1. Introduction to OOP

Object-Oriented Programming (OOP) is a programming approach that organizes code around objects. An object combines data (attributes) and behavior (methods). Python supports OOP through classes, objects, inheritance, encapsulation, polymorphism, and abstraction.

OOP is useful when a program models entities with state and behavior, such as students, bank accounts, products, or learning resources.

- Class: a definition or blueprint describing the attributes and methods its objects can have.
- Object: a particular instance created from a class.
- Attribute: data associated with an object or class.
- Method: a function defined inside a class.

2. Classes and Objects

A class defines a new type. An object is an instance of that class with its own state. For example, Student is a class, while a particular student named Asha is an object.

A class can contain attributes and methods. The dot operator accesses an object's attributes and methods.

- Student is the class.
- student1 and student2 are separate objects.
- The name attribute stores object-specific data.
- introduce() is a method that describes object behavior.

```
class Student:
    def introduce(self):
        print("I am", self.name)


student1 = Student()
student1.name = "Asha"
student1.introduce()

student2 = Student()
student2.name = "Ravi"
student2.introduce()
```

A better design with __init__

Attributes are commonly initialized in __init__ so every new object starts with the required data.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"{self.name} is {self.age} years old")


student = Student("Asha", 20)
student.introduce()
```

3. Constructor and self

In Python, __init__ is the initializer that runs automatically after a new instance is created. It is commonly called the constructor in beginner explanations, although __new__ is responsible for creating the instance itself.

self refers to the current instance. It is the first parameter of ordinary instance methods, and Python passes the object to it when the method is called through that object.

- self.brand is an instance attribute.
- brand is a local parameter received by __init__.
- self is a conventional parameter name, not a Python keyword.
- Do not write self explicitly when calling laptop.show(); Python supplies it.

```
class Laptop:
    def __init__(self, brand, ram):
        self.brand = brand
        self.ram = ram

    def show(self):
        print(self.brand, self.ram)


laptop = Laptop("Lenovo", "16 GB")
laptop.show()
```

4. Encapsulation

Encapsulation groups data and the methods that operate on it inside a class. It also provides a controlled interface for accessing or updating an object's state.

Python does not enforce traditional private access in the same way as some languages. A leading underscore signals that an attribute is intended for internal use. Double-leading underscores trigger name mangling.

- Use methods to validate changes instead of allowing arbitrary updates.
- _balance is a convention for internal use, not strict access control.
- Name mangling helps avoid accidental name clashes; it is not security.

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance


account = BankAccount(1000)
account.deposit(250)
print(account.get_balance())  # 1250
```

5. Inheritance

Inheritance allows a class (child or subclass) to reuse or extend behavior from another class (parent or superclass). Use inheritance when the child represents a meaningful specialized form of the parent.

- Dog inherits from Animal.
- The child class can add methods or override inherited methods.
- super() can call behavior from the parent class.

```
class Animal:
    def speak(self):
        print("Animal sound")
```

```
class Dog(Animal):
    def speak(self):
        print("Bark")


dog = Dog()
dog.speak()  # Bark
```

Using super()

```
class Person:
    def __init__(self, name):
        self.name = name


class Student(Person):
    def __init__(self, name, usn):
        super().__init__(name)
        self.usn = usn
```

6. Polymorphism

Polymorphism means that the same operation or interface can work with objects of different types. In Python, this often appears through method overriding and duck typing.

- Both objects provide speak(), so the loop can use them through the same interface.
- Python generally relies on supported behavior rather than requiring a shared declared interface.

```
class Cat:
    def speak(self):
        return "Meow"


class Dog:
    def speak(self):
        return "Bark"


for animal in (Cat(), Dog()):
    print(animal.speak())
```

7. Abstraction

Abstraction exposes essential operations while hiding implementation details. Python's abc module supports abstract base classes that define methods subclasses must implement.

- Shape defines the expected area() interface.
- Rectangle implements the abstract method.
- An abstract class with unimplemented abstract methods cannot be instantiated.

```
from abc import ABC, abstractmethod


class Shape(ABC):
    @abstractmethod
    def area(self):
        pass


class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
```

```
        self.height = height

    def area(self):
        return self.width * self.height


print(Rectangle(4, 3).area())  # 12
```

8. Class vs Instance Variables

A class variable is stored on the class and is shared by instances unless an instance shadows it. An instance variable belongs to a specific object.

- Use class variables for values intended to be shared.
- Use instance variables for per-object state.
- Mutating a shared mutable class variable can affect all instances that use it.

```
class Employee:
    company = "Acme"  # class variable

    def __init__(self, name):
        self.name = name  # instance variable


e1 = Employee("Asha")
e2 = Employee("Ravi")

print(e1.company, e2.company)
print(e1.name, e2.name)
```

9. Special Methods

Special methods, often called dunder methods because their names begin and end with double underscores, let user-defined objects participate in Python language operations.

- __init__: initialize a newly created instance.
- __str__: provide a readable string representation.
- __len__: define behavior for len(object).
- __repr__: provide a representation useful for debugging.

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} ({self.pages} pages)"

    def __len__(self):
        return self.pages


book = Book("Python Basics", 250)
print(book)
print(len(book))
```

10. Composition

Composition builds a class using objects of other classes. It represents a has-a relationship and is often more flexible than inheritance when one object simply uses another.

- A Car has an Engine.
- Car delegates engine behavior to its Engine object.
- Composition helps keep responsibilities separate.

```
class Engine:
    def start(self):
        print("Engine started")


class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()


car = Car()
car.start()
```

11. Interview Revision

Use these short comparisons for revision. Explain each concept with a definition and a small example rather than memorizing isolated phrases.

- Be ready to define a class, instantiate an object, and explain self.
- Trace __init__ and method calls in a short program.
- Explain inheritance, overriding, polymorphism, and super().
- Distinguish class variables from instance variables.
- Choose composition when one class uses another object rather than specializing it.

Class vs object

A class defines structure and behavior; an object is a concrete instance of that class.

__init__ vs self

__init__ initializes instance state; self refers to the current instance inside an instance method.

Inheritance vs composition

Inheritance models an is-a relationship; composition models a has-a relationship.

Overriding vs overloading

Overriding replaces inherited behavior in a subclass. Python does not support traditional signature-based method overloading in the same way as Java; common alternatives include default arguments and *args.

Encapsulation vs abstraction

Encapsulation organizes state and controls access. Abstraction presents a simpler interface while hiding implementation details.

Quick Practice Questions

1. What is the difference between a class and an object?
2. Why is self included in an instance method definition?
3. When does __init__ execute?

4. How does inheritance reduce code duplication?
5. What is method overriding?
6. How does encapsulation help protect object state?
7. What is the difference between a class variable and an instance variable?
8. Give an example of polymorphism in Python.
9. What problem do abstract base classes solve?
10. When is composition preferable to inheritance?